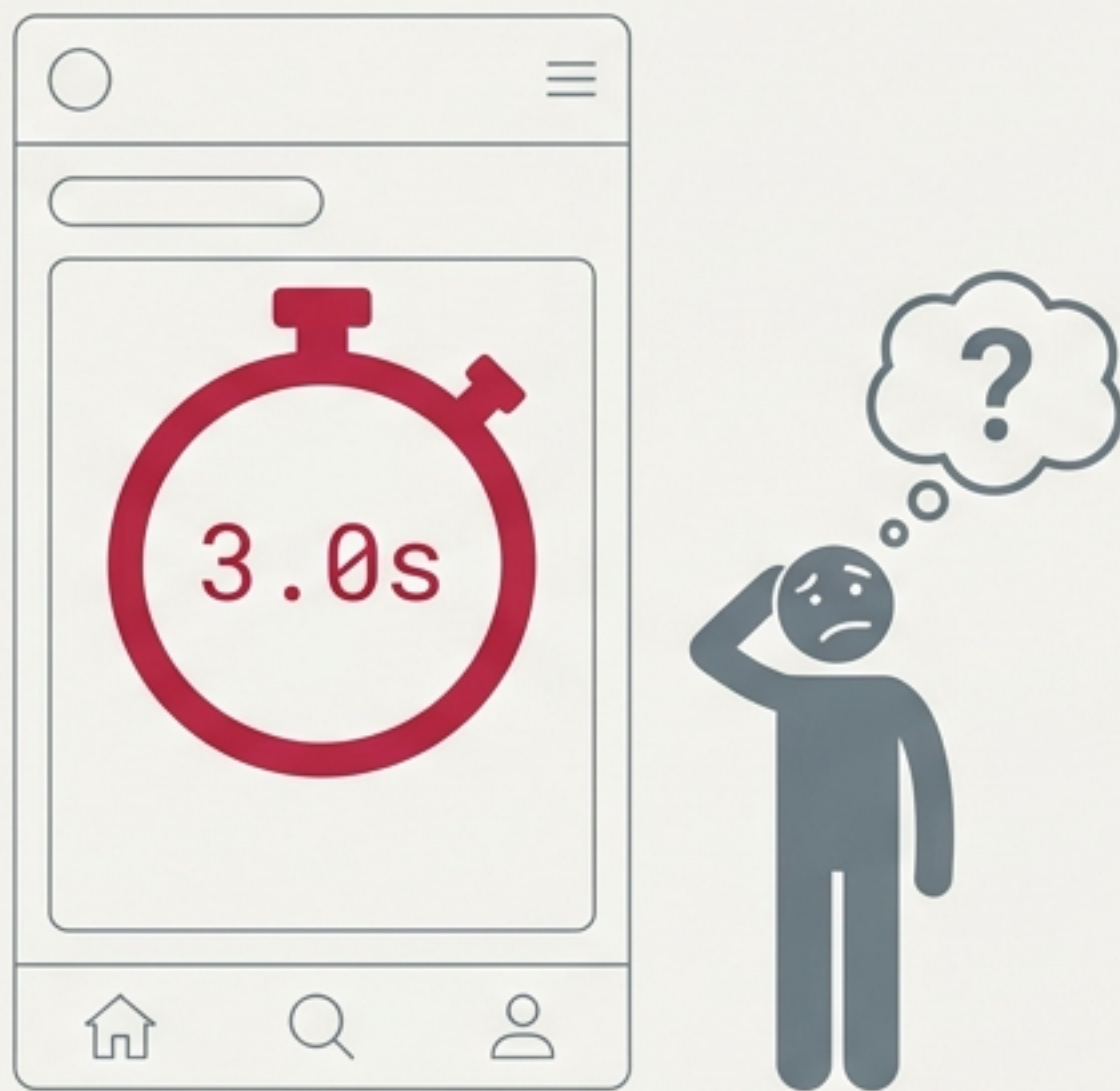


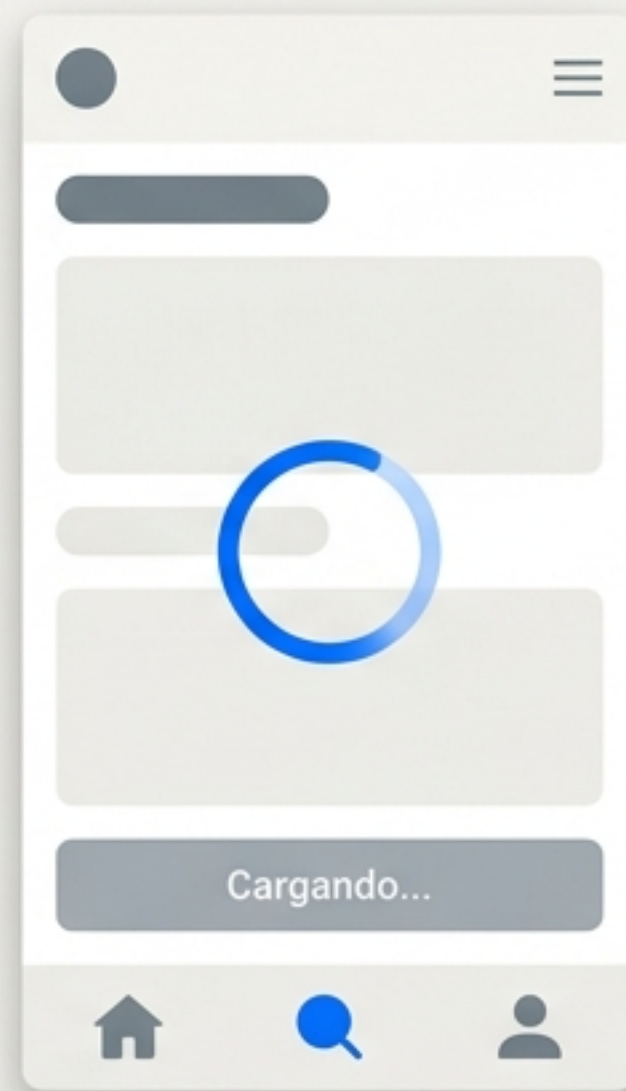
Lo que separa una app que “funciona” de una **interfaz profesional**

App Amateur



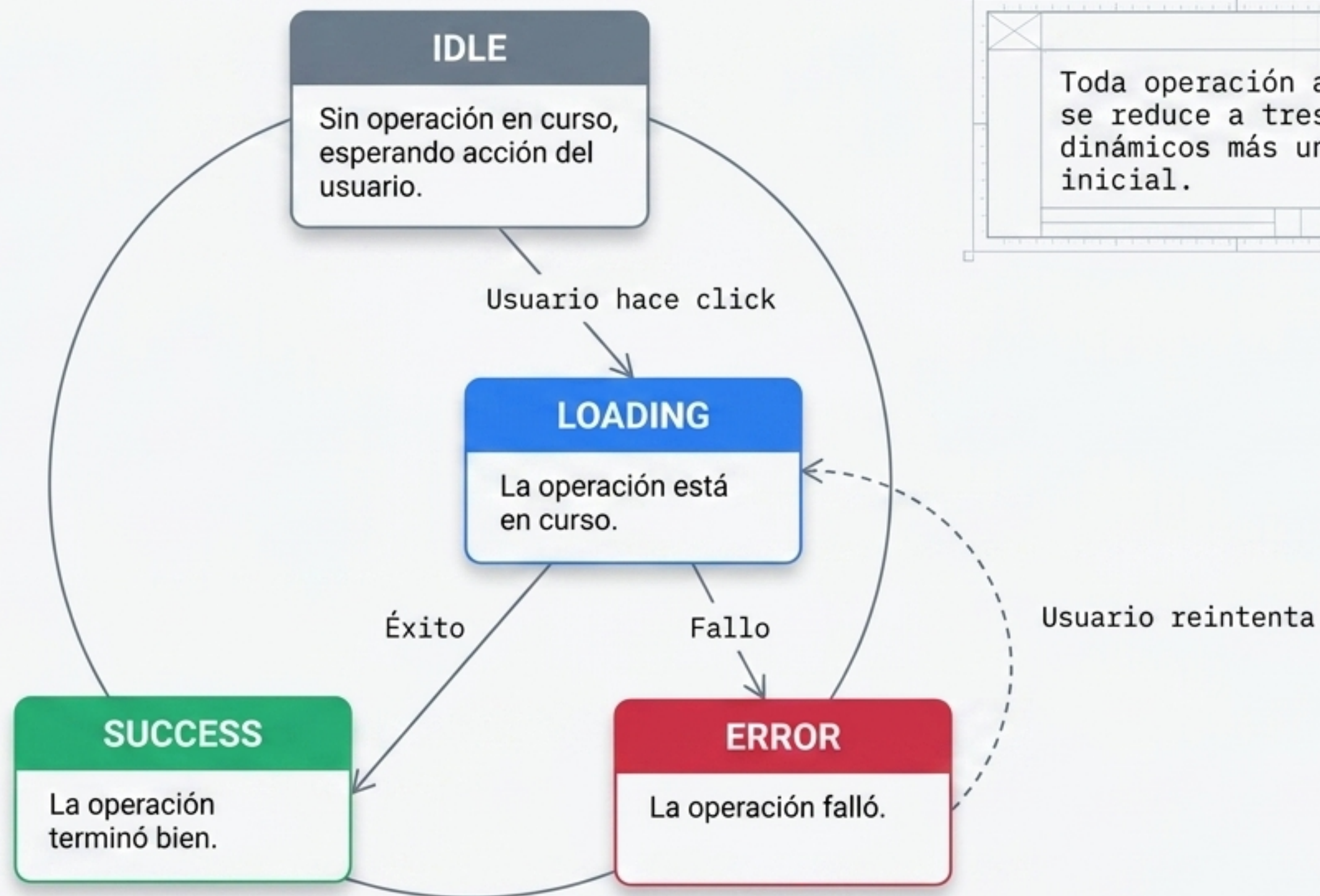
¿Mi click funcionó? ¿La app se colgó?
El usuario sufre la incertidumbre de
la red (latencia de 200ms a 3s).

App Profesional



El usuario recibe feedback inmediato.
Los estados de UI visualizan lo que
ocurre en el código durante la espera.

La Matriz Absoluta: Los 4 estados de toda operación asíncrona



Toda operación asíncrona se reduce a tres estados dinámicos más un estado inicial.

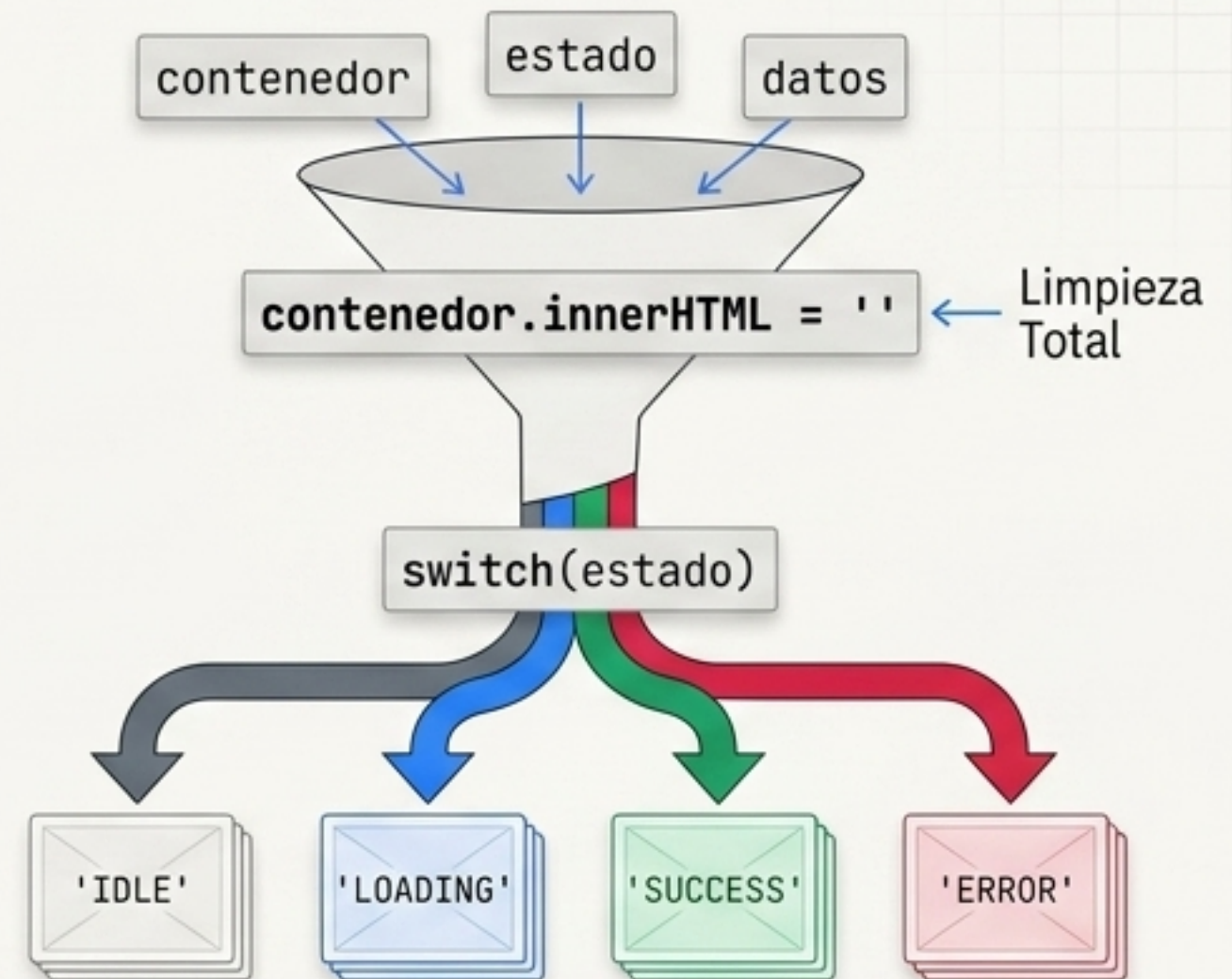
El patrón de estado único: Un "Director de Orquesta" para el DOM

El Mapa Lógico

```
const ESTADOS = {  
  IDLE: 'idle',  
  LOADING: 'loading',  
  ERROR: 'error',  
  SUCCESS: 'success'  
}
```

Usar un diccionario
estático previene
errores catastróficos
de tipeo.

El Motor de Renderizado

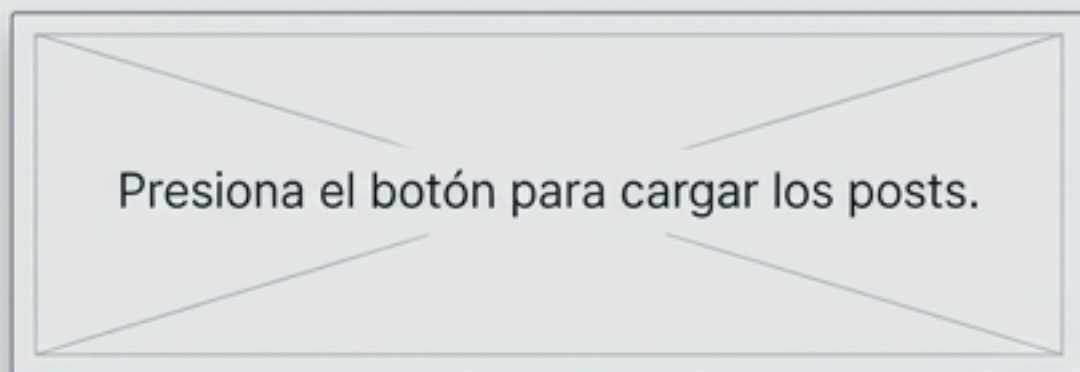


Una única función asume el control absoluto de la vista.

Inyección dinámica: Construyendo la interfaz caso por caso

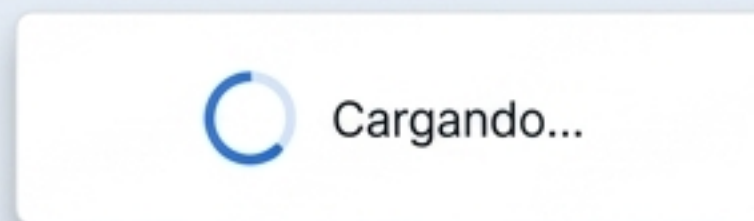
ESTADOS.IDLE

```
<p class="idle-msg">Presiona el botón...</p>
```



ESTADOS.LOADING

```
<div class="loading">  
  <div class="spinner"></div>  
  <p>Cargando...</p>  
</div>
```



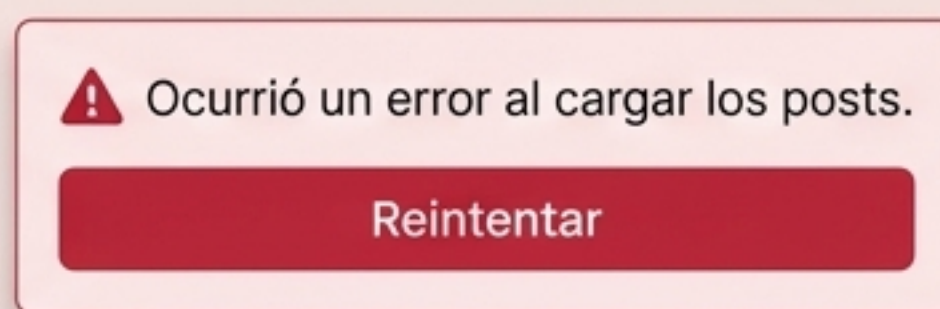
ESTADOS.SUCCESS

```
datos.forEach(post => {  
  contenedor.appendChild(crearCardPost(post))  
})
```



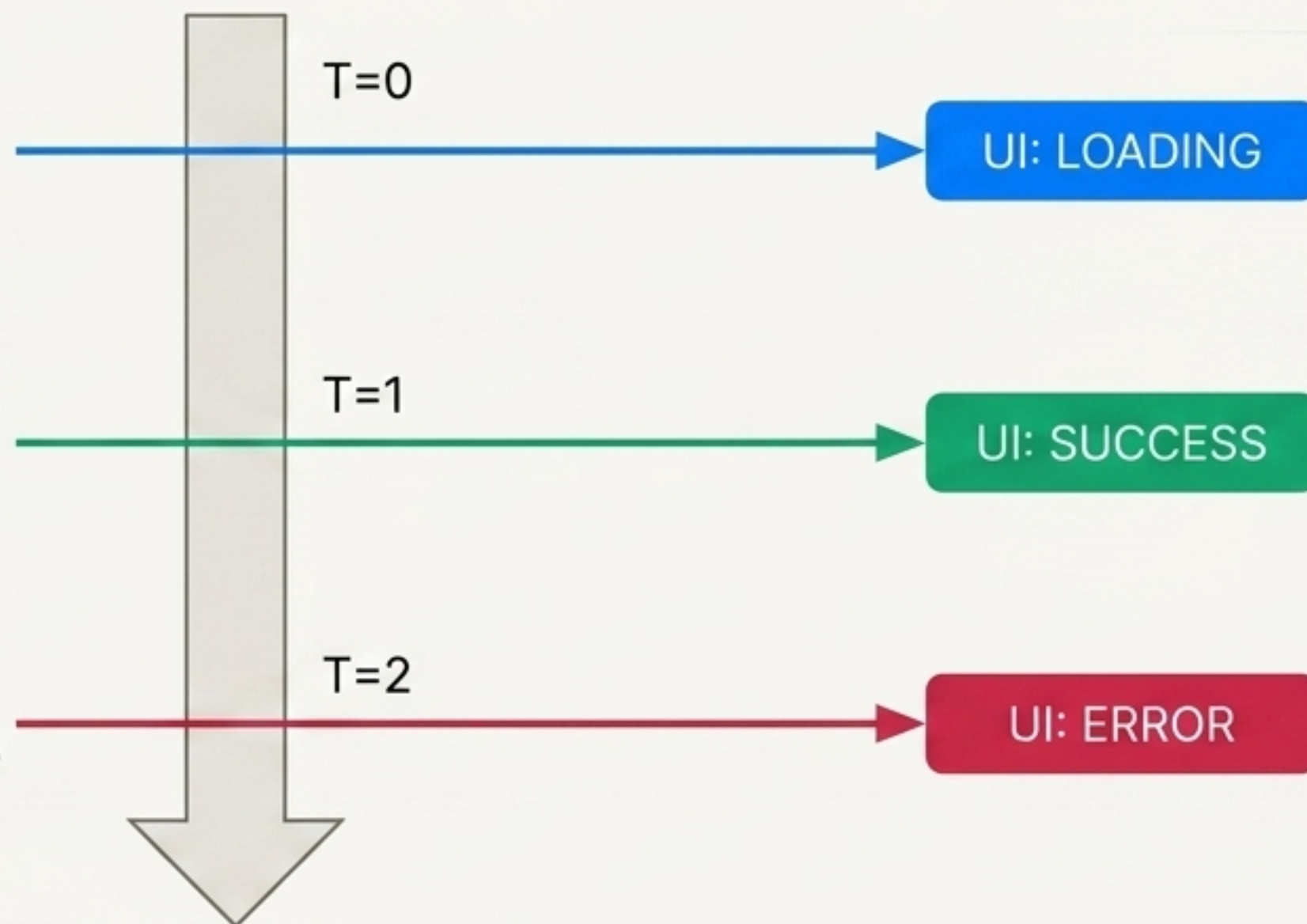
ESTADOS.ERROR

```
<p>⚠️ ${datos?.mensaje}</p>  
<button id="btn-reintentar">Reintentar</button>
```



Sincronizando la UI con el flujo temporal de fetch

```
async function cargarPosts() {  
  renderizarEstado(contenedor,  
    ESTADOS.LOADING)  
  
  try {  
    const data = await getPosts()  
    renderizarEstado(contenedor,  
      ESTADOS.SUCCESS, posts)  
  } catch (error) {  
    renderizarEstado(contenedor,  
      ESTADOS.ERROR, { mensajeerror.message  
    })  
  }  
}
```



Mapeo Exacto: Cada bloque de la asincronía (inicio, resolución, rechazo) dicta un repintado obligatorio del DOM.

Resiliencia: Manejando datos vacíos y bucles de reintento

Micro-patrón 1: El Estado Vacío

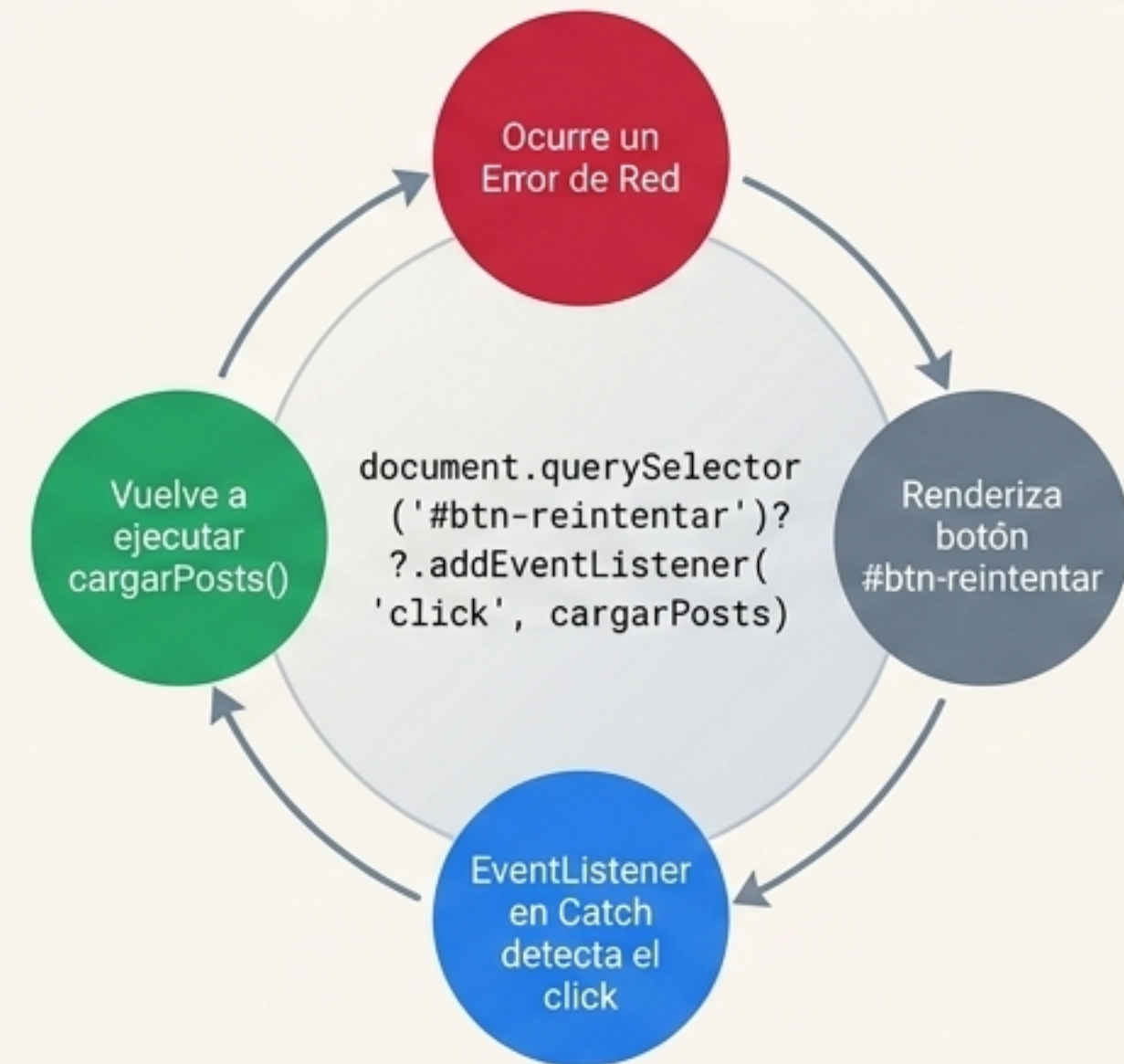
```
if (!datos || datos.length === 0) {  
  contenedor.innerHTML = '<p  
class="empty">No hay posts  
disponibles.</p>'  
  return  
}
```



No hay posts
disponibles.

Guardián de Éxito:
Previene que la UI se
rompa si la petición es
exitosa pero la base de
datos está vacía.

Micro-patrón 2: El Bucle de Recuperación



Anatomía del Spinner: Indicadores de carga en CSS puro

Geometría Base

```
width: 40px;  
height: 40px;  
border-radius: 50%;
```

Contraste Visual

```
border: 4px solid #f3f3f3;  
(Anillo estático)  
border-top: 4px solid #3498db;  
(Segmento activo)
```

Motor de Animación

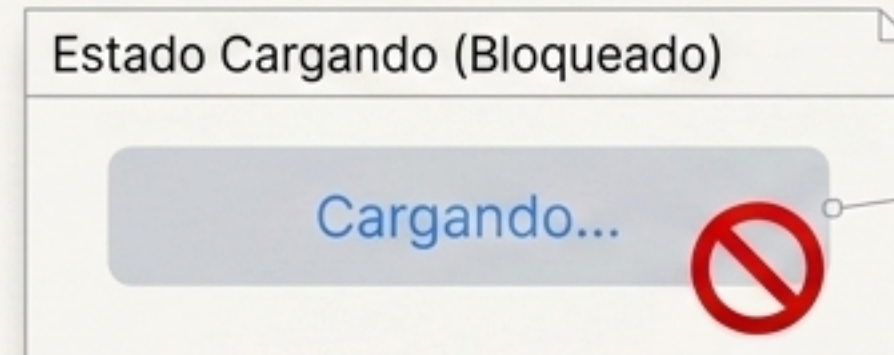
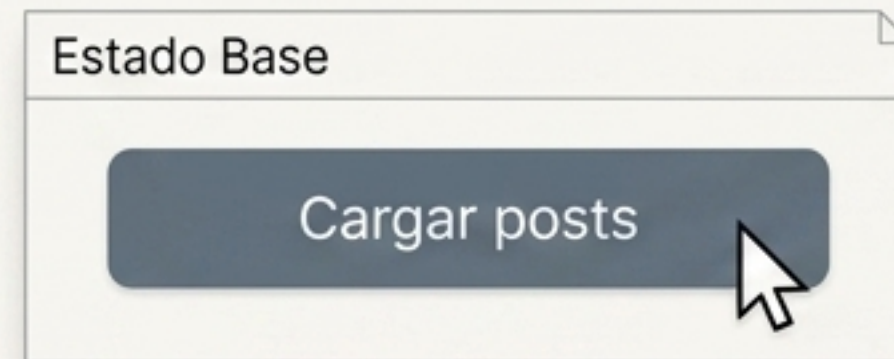
```
animation: girar 1s linear  
infinite;
```

```
@keyframes girar {  
  0% { transform: rotate(0deg); }  
  100% { transform: rotate(360deg); }  
}
```

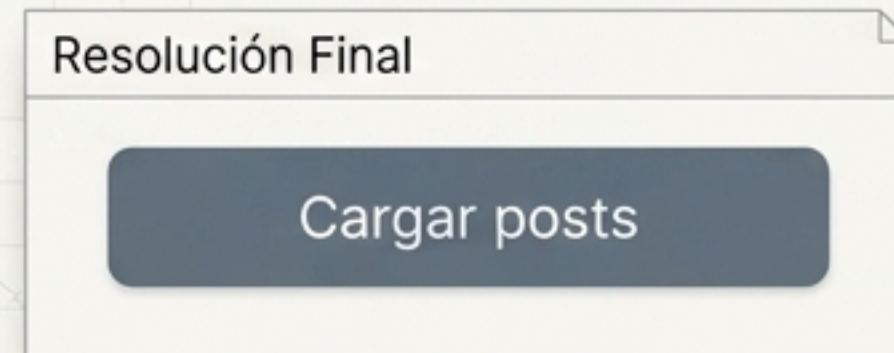
Insight: Construirlo en CSS puro elimina el tiempo de carga de red de descargar un GIF o SVG.

El Botón Comunicador: Previniendo la duplicación de peticiones

Simulación de Estados



disabled = true.
Previene el temido
doble-click.



Control Imperativo

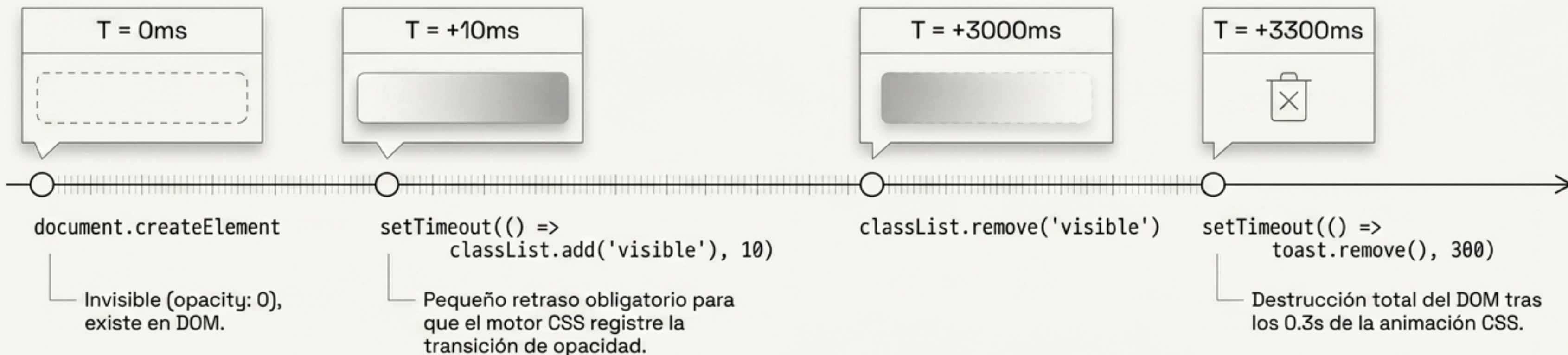
```
function setEstadoBoton(boton, cargando) {  
  boton.disabled = cargando  
  boton.textContent = cargando ? 'Cargando...' : 'Cargar posts'  
}
```

```
try {  
  ...await getPosts()  
} catch (error) {  
  ...  
} finally {  
  setEstadoBoton(boton, false) // Siempre restaurar  
}
```

El bloque finally garantiza que el botón se restaure independientemente de si hubo éxito o error.

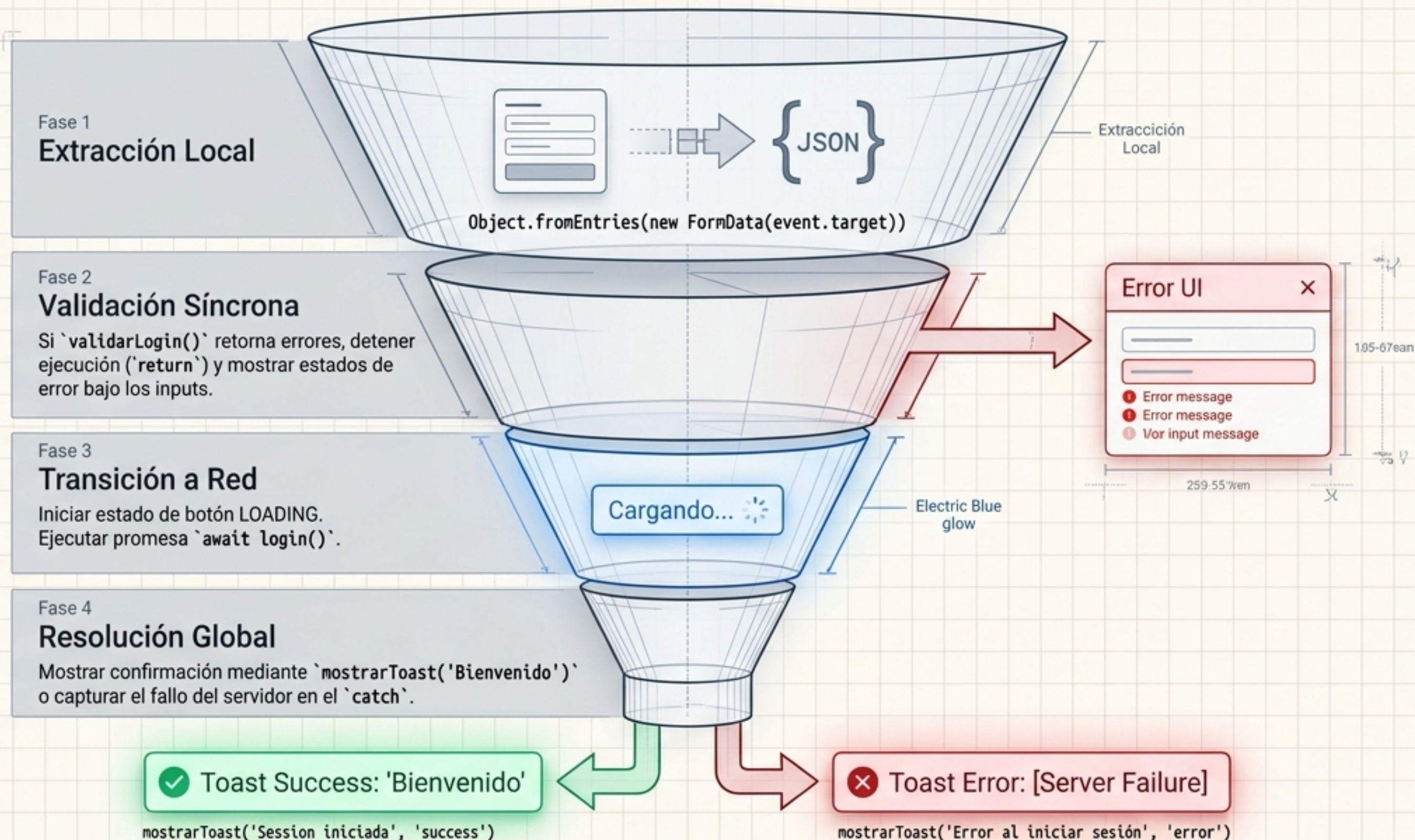
Mensajería Efímera: El ciclo de vida de un Toast

Timing Function



```
.toast-success {  
  background: #16a34a;  
}  
  
.toast-error {  
  background: #dc2626;  
}
```


Integración compleja: Formulario de Login Multi-Estado



Estrategia de diagnóstico: Errores Locales vs. Globales

Error Local
(Específico de campo)



Generado antes del fetch.
Iteración sobre
``Object.entrieserrores)``.
Ayuda al usuario a corregir un
typo inmediatamente mediante
``mostrarErrorCampo()``.

Login Form

Email

Password

Submit

Error característico en el
código también en la
comunicación de fallos observados a un
host del espacio volverá
mensaje.

Error Global
(Servidor / Red)

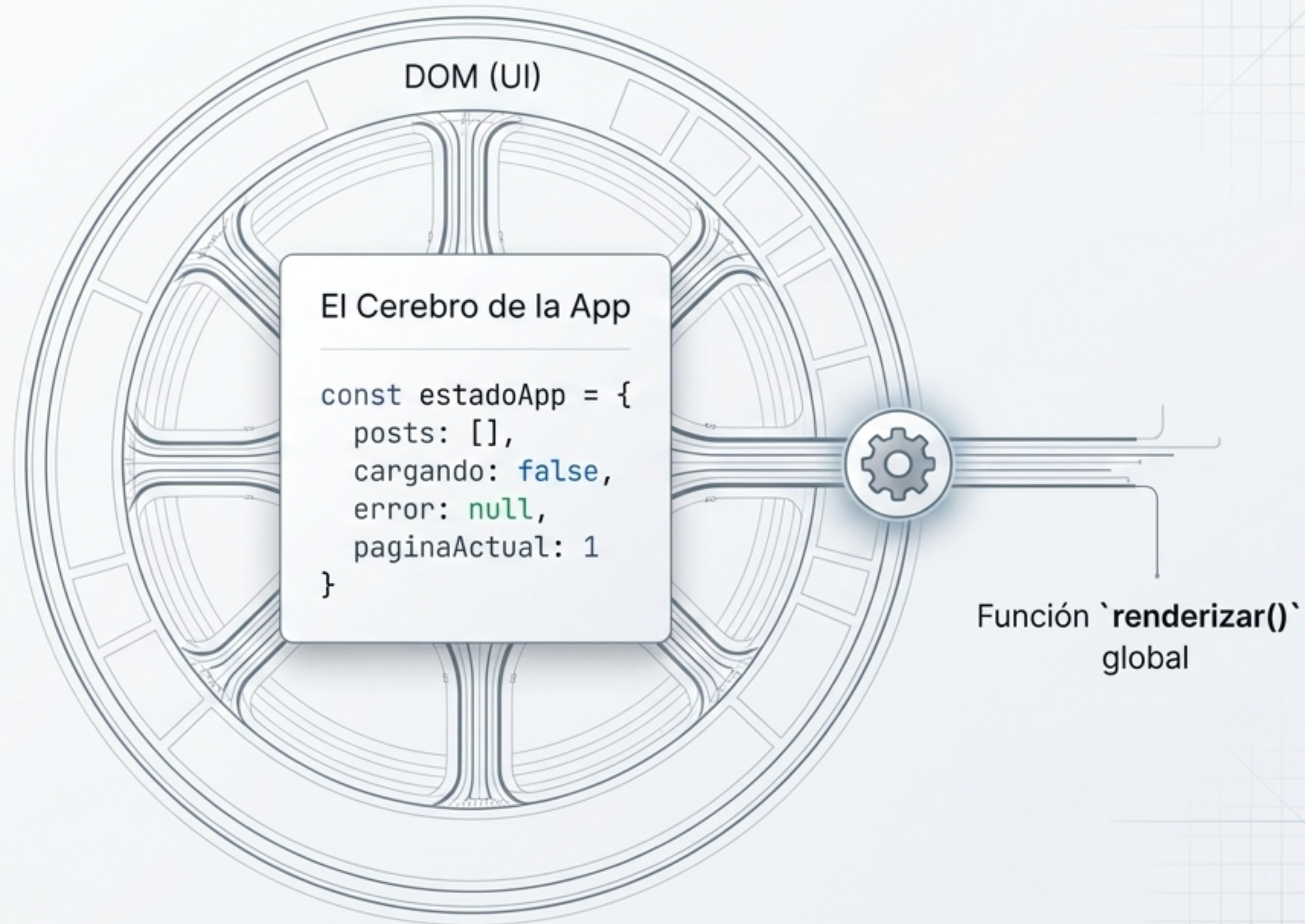
❗ Servidor no disponible

Generado en el bloque ``catch``.
Comunica fallos de infraestructura fuera
del control del formulario mediante
``mostrarErrorGeneral(error.message)``.

El Santo Grial: Arquitectura de Estado Centralizado

El Flujo Modificado:

- ✓ Se abandona la manipulación imperativa y dispersa del DOM.
- ✓ Funciones como `cargarPosts()` ya no inyectan HTML directamente. Únicamente mutan las propiedades de `estadoApp`.
- ✓ Tras cada mutación, se invoca ciegamente `renderizar()`, la cual evalúa el objeto y delega el dibujo de la vista.



El Puente hacia el Futuro: React y la automatización del patrón

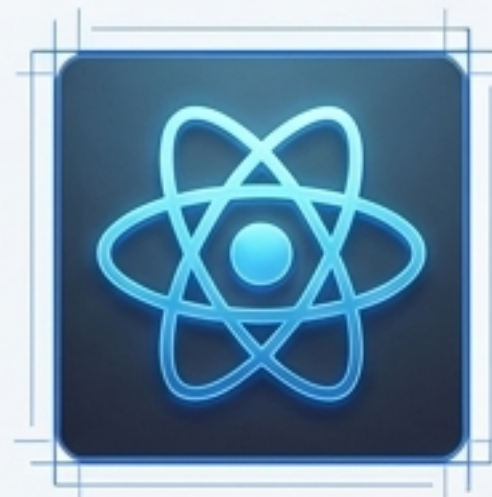
Lo que construiste hoy (Vanilla JS)

1. Creas el objeto ``estadoApp``.
2. Modificas manualmente la propiedad (ej. ``estadoApp.cargando = true``).
3. Invocas manualmente ``renderizar()`` para repintar el DOM.



Lo que usarás mañana (React)

1. Invocas ``const [cargando, setCargando] = useState(false)``.
2. Ejecutas ``setCargando(true)``.
3. Magia: React repinta la UI automáticamente al detectar el cambio.



The Takeaway: Has construido a mano el patrón de re-renderizado centralizado. Cuando React automatice esto por debajo, comprenderás exactamente cómo la lógica dicta la experiencia del usuario.